Algoritmos Avançados

Aula 16: Introdução à Teoria dos Grafos & Representações Computacionais

Prof. Me. Douglas Samuel Gonçalves

Fatec Bebedouro – Jorge Caram Sabbag
Tecnologia em Big Data no Agronegócio

2º Semestre / 2026

Sumário da Aula

1. Motivação Histórica & Relevância no Big Data
2. Definições Formais & Terminologia Essencial
3. Caminhos, Ciclos & Topologia Estrutural
4. Representações Computacionais em Memória
5. Implementação Orientada a Objetos em Python
6. Estudo de Caso Prático: Malha Logística no Agro
7. Exercícios Práticos Resolvidos & Desafios
8. Próximos Passos & Encerramento

O Ponto de Partida: A Cidade de Königsberg (1736)

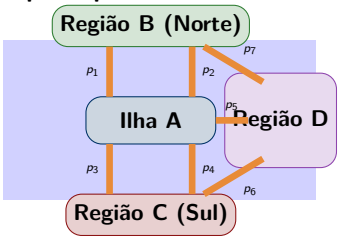
O Enigma Histórico:

- A cidade prussiana de **Königsberg** (atual Kaliningrado) era cortada pelo rio Pregel.
- O rio circundava a ilha **A** (Kneiphof) e dividia a cidade em quatro regiões distintas de terra: **A**, **B**, **C** e **D**.
- Sete pontes interligavam essas regiões.

O Desafio dos Moradores

É possível realizar um passeio pela cidade partindo de qualquer ponto, cruzando cada uma das sete pontes exatamente uma vez, e retornando ao ponto inicial?

Mapa Esquemático das 7 Pontes



A Solução Genial de Leonhard Euler (1736)

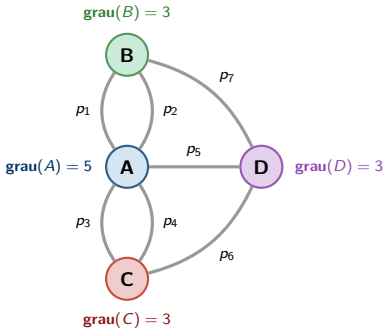
A Sacada Teórica Fundamental:

- Euler percebeu que **o formato da ilha**, **o relevo** e **o comprimento das pontes** eram irrelevantes para resolver o problema.
- A essência era puramente **estrutural e topológica**:
  - Regiões de terra $\longrightarrow$ **Vértices (Nós)**;
  - Pontes $\longrightarrow$ **Arestas (Conexões)**.

O Teorema de Euler

Para entrar e sair de uma região sem repetir pontes, cada visita consome **duas conexões** (entrada e saída). Logo, a quantidade de pontes ligadas a cada região deve ser **par**!

O Grafo Abstrato de Königsberg



Todos os graus são ímpares $\Longrightarrow$ Passeio impossível!

Por que Grafos em Big Data & Agronegócio?

A Teoria dos Grafos é a estrutura matemática ideal para modelar **entidades** e suas **relações interconectadas**.

Logística & Cadeia Agro

- **Vértices:** Talhões, silos de transbordo, usinas e portos.
- **Arestas:** Rodovias com frete, pedágio e distância ponderada.
- **Objetivo:** Roteirização de frotas e menor custo.

Telemetria & IoT Mesh

- **Vértices:** Sensores de solo, pivôs de irrigação.
- **Arestas:** Enlaces sem fio (LoRaWAN, 4G/5G).
- **Objetivo:** Tolerância a falhas na lavoura.

Pipelines de Dados (DAGs)

- Dependências de tarefas ETL em orquestradores modernos (Apache Airflow, Spark).

Rastreabilidade & Redes

- Dispersão epidemiológica de pragas na cultura.
- Grafos de Conhecimento e crédito rural.

Definição Formal de um Grafo

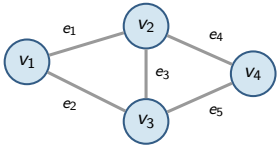
Definition (Grafo)

Um **Grafo** G é um par ordenado $G = (V, E)$, composto por:

- Um conjunto finito e não-vazio $V = \{v_1, v_2, \ldots, v_n\}$ de elementos denominados **Vértices** (ou Nós);
- Um conjunto $E = \{e_1, e_2, \ldots, e_m\}$ de pares de vértices, denominados **Arestas** (ou Arcos / Conexões).

Notações e Dimensões do Grafo

- **Ordem (n):** Número total de vértices $= |V|$.
- **Tamanho (m):** Número total de arestas $= |E|$.
- Uma aresta e entre u e v: $e = (u, v)$ ou $e = \{u, v\}$.



$V = \{v_1, v_2, v_3, v_4\}$, $|V| = 4$

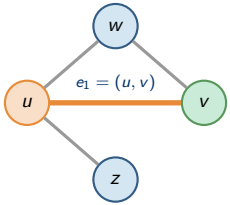
$E = \{e_1, e_2, e_3, e_4, e_5\}$, $|E| = 5$

Conceitos Fundamentais: Adjacência, Incidência e Vizinhos

Relações Estruturais

- **Adjacência:** Dois vértices $u, v \in V$ são **adjacentes** (vizinhos) se existe uma aresta $e = (u, v) \in E$.
- **Incidência:** Uma aresta $e = (u, v)$ é dita **incidente** aos vértices u e v (suas extremidades).
- **Arestas Adjacentes:** Arestas que compartilham ao menos um vértice em comum.
- **Vizinhança $N(v)$:** Conjunto de vértices diretamente conectados a v:

$$N(v) = \{u \in V \mid (v, u) \in E\}$$



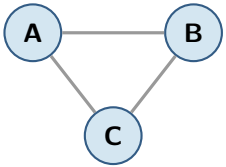
Leitura do Diagrama

- u e v são **adjacentes**;
- Aresta e_1 é **incidente** a u e v;
- Vizinhança de u: $N(u) = \{v, w, z\}$.

Classificação: Não-Direcionados vs. Direcionados (Digrafos)

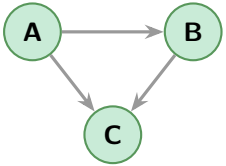
A orientação das arestas define o fluxo da informação ou matéria dentro da estrutura.

Grafo Não-Direcionado (Arestas Simétricas)



- As arestas são **pares não-ordenados**: $(u, v) \equiv (v, u)$.
- Conexão de mão dupla (bidirecional).
- **Exemplo:** Rede de amizades no LinkedIn, rodovia asfaltada convencional de sentido duplo.

Grafo Direcionado / Digrafo (Arcos com Sentido)

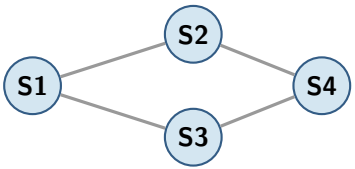


- As arestas são **pares ordenados**: $(u, v) \neq (v, u)$.
- u é a **origem** e v é o **destino**.
- **Exemplo:** Seguidor no Twitter/X, fluxo de água em duto de irrigação, transações bancárias Pix.

Classificação: Não-Ponderados vs. Ponderados (Valorados)

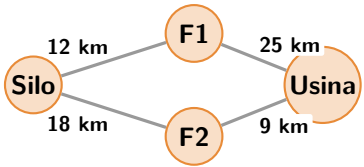
Em cenários reais de Big Data e Engenharia, conexões possuem intensidades ou custos associados.

Grafo Não-Ponderado (Topologia Pura)



- A aresta apenas indica a **existência ou não** de um enlace.
- Todas as conexões possuem o mesmo custo implícito ($w = 1$).
- **Exemplo:** Rede de computadores comunicando-se via broadcast.

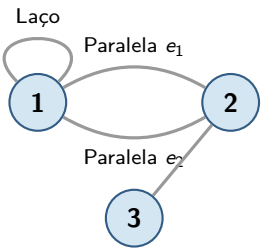
Grafo Ponderado (Com Pesos / Custos)



- Cada aresta e recebe uma função de peso $w : E \to \mathbb{R}$.
- Representa: distância (km), frete (R$), tempo (min), capacidade (toneladas/h).
- **Exemplo:** Otimização do escoamento da safra de cana-de-açúcar.

Grafos Simples, Multigrafos e Laços

Nem todos os grafos respeitam a restrição de uma única aresta por par de vértices.



Definições Específicas

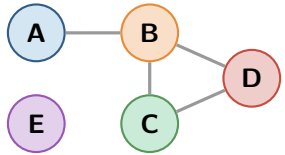
- **Laço (Self-loop):** Aresta onde origem e destino coincidem no mesmo vértice: $e = (v, v)$.
- **Arestas Paralelas:** Duas ou mais arestas distintas unindo o mesmo par de vértices.
- **Grafo Simples:** Sem laços nem arestas paralelas. Padrão na disciplina!
- **Multigrafo:** Permite arestas paralelas.
- **Pseudografo:** Permite arestas paralelas e laços.

O Conceito de Grau de um Vértice

Definition (Grau $\deg(v)$)

Quantidade de arestas incidentes a v.

- Laços contam **duas vezes** no grau.



$\deg(B) = 3$ (Hub), $\deg(C) = \deg(D) = 2$,
$\deg(A) = 1$ (Pendente), $\deg(E) = 0$ (Isolado).

Nomenclaturas de Destaque

- **Vértice Isolado:** $\deg(v) = 0$.
- **Vértice Pendente (Folha):** $\deg(v) = 1$.
- **Grau Mínimo ($\delta(G)$):** Menor grau de G.
- **Grau Máximo ($\Delta(G)$):** Maior grau de G.

$$\delta(G) \leq \deg(v) \leq \Delta(G), \quad \forall v \in V$$

O Teorema Fundamental: Lema do Aperto de Mãos

Um dos resultados mais elegantes e úteis de toda a Teoria dos Grafos.

Teorema (Handshaking Lemma – Euler)

Em qualquer grafo não-direcionado $G = (V, E)$, a soma dos graus de todos os seus vértices é exatamente igual ao dobro do número de arestas:

$$\sum_{v \in V} \deg(v) = 2|E|$$

Intuição Visual da Prova:

- Cada aresta $e = (u, v)$ possui rigorosamente **duas extremidades**.
- Ao somar os graus de todos os vértices, cada aresta contribui com $+1$ em u e $+1$ em v.
- Logo, cada aresta é contada exatamente 2 vezes!

Corolário Inevitável

Em qualquer grafo, **o número de vértices de grau ímpar é sempre PAR**.

Exemplo: Não existe rede com 3 pessoas onde cada uma tenha exatamente 3 amigos ($\sum \deg = 9$, ímpar!).

Graus em Digrafos: Entrada ($\deg^-$) e Saída ($\deg^+$)

Em grafos direcionados, o sentido dos arcos divide o grau em dois componentes vitais:

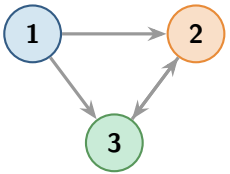
Definições

- **Grau de Entrada ($\deg^-(v)$):** Arcos que **chegam** a v.
- **Grau de Saída ($\deg^+(v)$):** Arcos que **partem** de v.
- **Grau Total:** $\deg(v) = \deg^-(v) + \deg^+(v)$.

Propriedade de Conservação

A soma dos graus de entrada = saída = $|E|$:

$$\sum_{v \in V} \deg^-(v) = \sum_{v \in V} \deg^+(v) = |E|$$



V	$\deg^-$	$\deg^+$	deg
1	0 (Fonte)	2	2
2	2	1	3
3	2	1	3
Σ	**4**	**4**	**8**

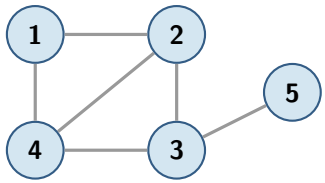
$|E| = 4$ arcos. Perfeita consonância!

Passeios, Trilhas e Caminhos

Termos tratados como sinônimos na linguagem informal, mas com **significados formais distintos** em Ciência da Computação:

A Escada da Restrição

- **Passeio (Walk):** Sequência alternada de vértices e arestas $v_0, e_1, v_1, e_2, \ldots, v_k$. Pode repetir vértices e arestas à vontade!
- **Trilha (Trail):** Um passeio onde **nenhuma aresta é repetida** (vértices podem se repetir). O problema de Königsberg exigia uma trilha euleriana fechada!
- **Caminho Simples (Path):** Um passeio onde **nenhum vértice é repetido** (e consequentemente nenhuma aresta).



- Passeio: $\langle 1, 2, 4, 1, 2, 3 \rangle$ (repete vértices e arestas).
- Trilha: $\langle 1, 2, 4, 3, 2 \rangle$ (repete vértice 2, arestas únicas).
- **Caminho Simples:** $\langle 1, 2, 3, 5 \rangle$ (todos os vértices distintos).

Ciclos e Grafos Acíclicos (DAGs)

Ciclo Simples

Caminho fechado de comprimento ≥ 3, origem = destino ($v_0 = v_k$), sem repetição de vértices intermediários.

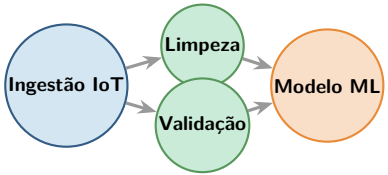
- Grafo sem nenhum ciclo $\rightarrow$ **Acíclico**.

DAG – Directed Acyclic Graph

Digrafo **sem ciclos orientados**.

- Estrutura central de escalonamento de tarefas.
- Admite **Ordenação Topológica**.

Pipeline de Dados Agro (DAG)



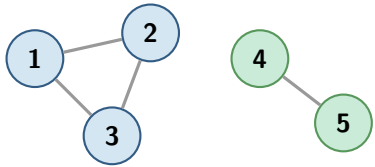
Fluxo estritamente unidirecional e livre de deadlocks!

Conectividade em Grafos Não-Direcionados

Definition (Grafo Conexo)

Um grafo não-direcionado é **Conexo** se existe pelo menos um caminho entre qualquer par de vértices $u, v \in V$. Caso contrário, é **Desconexo**.

Grafo Desconexo (2 Componentes Conexos)



Não existe caminho entre 1 e 5. O grafo possui 2 componentes conexos.

Aplicações Práticas

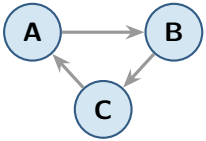
- **Rede de Sensores:** Se houver mais de 1 componente, parte dos sensores está incomunicável com a central.
- **Ilhas de Distribuição:** Identificar frentes de colheita isoladas por estradas intransitáveis.
- BFS conta e rotula todos os componentes em $O(V + E)$.

Conectividade em Digrafos: Forte vs. Fraca

Em grafos direcionados, a existência de caminhos depende da direção das setas.

Fortemente Conexo (Strongly Connected)

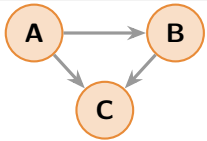
Existe um caminho orientado de u para v **E** também de v para u, para qualquer par de vértices $u, v \in V$.



Todos conseguem alcançar todos.

Fracamente Conexo (Weakly Connected)

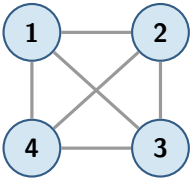
O digrafo só se torna conexo se **ignorarmos a direção** das setas (substituindo arcos direcionados por arestas simples).



Do nó C é impossível alcançar A ou B!

Famílias Especiais de Grafos: Completos e Bipartidos

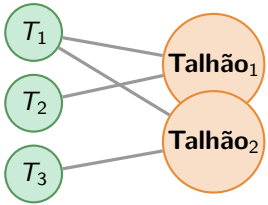
Grafo Completo (K_n)



- Cada par de vértices possui uma aresta direta.
- Número total de arestas de K_n:

$$|E| = \frac{n(n-1)}{2} = \Theta(n^2)$$

Grafo Bipartido ($G = (X \cup Y, E)$)



- Vértices particionados em dois grupos disjuntos X e Y.
- **Arestas só existem entre X e Y** (nunca internamente).
- Se houver todas as conexões possíveis, é **Bipartido Completo** ($K_{m,n}$).

Árvores sob a Perspectiva de Grafos

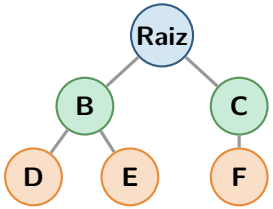
Nas aulas anteriores estudamos Árvores como estruturas hierárquicas. Na Teoria dos Grafos:

Definition (Árvore)

Uma **Árvore** é um grafo não-direcionado, **conexo** e **acíclico**.

Propriedades Fundamentais:

1. Possui exatamente $|V| - 1$ **arestas**: $|E| = |V| - 1$
2. Existe um **único caminho simples** entre qualquer par de vértices.
3. Remover qualquer aresta $\rightarrow$ grafo desconexo!
4. Adicionar qualquer aresta $\rightarrow$ cria um ciclo!



$|V| = 6 \implies |E| = 5$.
Conexo e 100% livre de ciclos.

O Dilema de Engenharia de Software & Big Data

Como representar um grafo na memória RAM do computador?

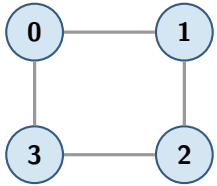
- No mundo matemático, representamos como $G = (V, E)$.
- No computador, precisamos de uma estrutura que responda rapidamente a perguntas como:
  - *"Os vértices u e v são vizinhos diretos?"*
  - *"Quais são todos os vizinhos do vértice u?"*
  - *"Qual é o peso da aresta entre u e v?"*

As Duas Representações Canônicas

1. **Matriz de Adjacência:** Uma tabela bidimensional $V \times V$ onde cada célula (i, j) indica a presença ou peso da aresta.
2. **Lista de Adjacência:** Um vetor (ou dicionário) de listas, onde cada vértice mantém a relação explícita de seus vizinhos.

Representação 1: Matriz de Adjacência

Grafo de Exemplo ($|V| = 4$)



Grafo não-direcionado e não-ponderado.

Matriz de Adjacência $A \in \{0,1\}^{4\times 4}$

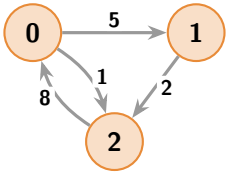
$$A = \begin{pmatrix} & \mathbf{0} & \mathbf{1} & \mathbf{2} & \mathbf{3} \\ \mathbf{0} & 0 & 1 & 0 & 1 \\ \mathbf{1} & 1 & 0 & 1 & 0 \\ \mathbf{2} & 0 & 1 & 0 & 1 \\ \mathbf{3} & 1 & 0 & 1 & 0 \end{pmatrix}$$

Propriedades da Matriz

- Não-direcionado $\Longrightarrow$ Matriz **Simétrica** ($A[i][j] = A[j][i]$).
- Diagonal $A[i][i] = 0$ (em grafos simples sem laços).
- Soma da linha i é igual ao grau $\deg(i)$ do vértice.

Matriz de Adjacência com Pesos e Digrafos

Digrafo Ponderado



Matriz Ponderada

$$A = \begin{pmatrix} & \mathbf{0} & \mathbf{1} & \mathbf{2} \\ \mathbf{0} & 0 & 5 & 1 \\ \mathbf{1} & \infty & 0 & 2 \\ \mathbf{2} & 8 & \infty & 0 \end{pmatrix}$$

- **Ausência de Aresta:** Usualmente preenche-se com ∞ (infinito) em algoritmos de caminho mínimo (ou 0 / `None`).
- **Assimetria:** Em digrafos, $A[i][j] \neq A[j][i]$ em geral ($A[0][1] = 5$, mas $A[1][0] = \infty$).

Avaliação Crítica da Matriz de Adjacência

Pontos Fortes (Vantagens)

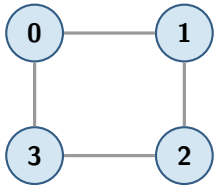
- **Consulta de Aresta Instantânea:** Testar se existe a aresta (u, v) custa $\mathbf{O(1)}$: basta acessar `M[u][v]`.
- **Inserção/Remoção de Aresta:** Operação imediata em $\mathbf{O(1)}$.
- **Álgebra Linear:** Permite calcular matrizes de potências (A^k indica a quantidade de passeios de comprimento k entre os vértices!).

Pontos Fracos (Desvantagens)

- **Custo Espacial Quadrático:** Ocupa invariavelmente $\mathbf{\Theta}(|\mathbf{V}|^2)$ de memória, mesmo que o grafo tenha pouquíssimas arestas!
- **Listar Vizinhos é Lento:** Para encontrar os vizinhos de u, precisamos obrigatoriamente iterar por toda a linha: $\mathbf{O}(|\mathbf{V}|)$.
- **Inserção de Vértice:** Operação cara de redimensionamento $\mathbf{O}(|\mathbf{V}|^2)$.

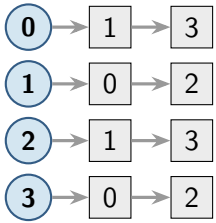
Representação 2: Lista de Adjacência

Grafo de Exemplo ($|V| = 4$)



Cada vértice possui uma lista exclusiva contendo **apenas os seus vizinhos reais**.

Estrutura de Listas / Dicionário



Em Python: `{0: [1, 3], 1: [0, 2], 2: [1, 3], 3: [0, 2]}`

Avaliação Crítica da Lista de Adjacência

Pontos Fortes (Vantagens)

- **Eficiência Espacial Imbatível:** Ocupa apenas $\mathbf{O}(|\mathbf{V}| + |\mathbf{E}|)$ de memória.
- **Ideal para Grafos Esparsos:** Onde o número de conexões é muito menor que o limite teórico ($|E| \ll |V|^2$).
- **Iteração Rápida sobre Vizinhos:** O tempo para percorrer os vizinhos de u é proporcional ao seu grau real: $\mathbf{O}(\deg(\mathbf{u}))$.
- **Adicionar Vértice:** Instantâneo em $\mathbf{O}(\mathbf{1})$.

Pontos Fracos (Desvantagens)

- **Consulta de Aresta:** Para verificar se $(u, v) \in E$, precisamos buscar v dentro da lista de vizinhos de u: tempo $\mathbf{O}(\deg(\mathbf{u}))$ (ou $O(1)$ se usarmos conjuntos `set` ou dicionários como tabelas hash em Python!).
- **Remoção de Aresta:** Requer busca linear na lista encadeada.

Representação 3: Lista de Arestas (Edge List)

Uma terceira alternativa muito comum em algoritmos gulosos de ordenação de conexões (como o Algoritmo de Kruskal).

Estrutura

Consiste em um vetor ou lista simples contendo todas as arestas do grafo como tuplas:

$$E = [(u_1, v_1, w_1), (u_2, v_2, w_2), \dots]$$

- Ocupa exatamente $\mathbf{O}(|\mathbf{E}|)$ de memória.
- Excelente para ordenar arestas por peso (ex: do menor para o maior frete).

Exemplo em Python

```python
# Lista de Arestas: (Origem, Destino, Distancia_km)
arestas_agro = [
    ("Silo_1", "Usina", 14.5),
    ("Silo_2", "Usina", 22.0),
    ("Talhao_A", "Silo_1", 4.2),
    ("Talhao_B", "Silo_2", 8.1)
]

# Ordenacao rapida por custo/peso
arestas_ordenadas = sorted(
    arestas_agro, key=lambda x: x[2]
)
```

Tabela Comparativa de Complexidade Assintótica

Operação / Métrica	Matriz de Adjacência	Lista de Adjacência	Lista de Arestas
Memória (Espaço)	$\mathbf{\Theta(\lvert V\rvert^2)}$	$\mathbf{\Theta(\lvert V\rvert + \lvert E\rvert)}$	$\mathbf{\Theta(\lvert E\rvert)}$
Verificar $(u, v) \in E$	$\mathbf{O(1)}$	$O(\deg(u))$ *	$O(\lvert E\rvert)$
Listar vizinhos de u	$O(\lvert V\rvert)$	$\mathbf{O(\deg(u))}$	$O(\lvert E\rvert)$
Inserir nova Aresta	$\mathbf{O(1)}$	$\mathbf{O(1)}$	$\mathbf{O(1)}$
Remover Aresta	$\mathbf{O(1)}$	$O(\deg(u))$	$O(\lvert E\rvert)$
Inserir novo Vértice	$O(\lvert V\rvert^2)$ (cópia)	$\mathbf{O(1)}$	$\mathbf{O(1)}$

* Em Python, utilizando dicionários com tabelas hash para os vizinhos (`dict[u][v]`), a consulta de adjacência atinge **tempo médio** $O(1)$, unindo o melhor dos dois mundos!

O Choque de Realidade em Big Data: O Teste da Memória

Rede de sensores agrícolas com $|V| = 10^6$ nós e média de 10 conexões por nó ($|E| = 5 \times 10^6$):

Cenário A: Matriz de Adjacência

- Células: $|V|^2 = 10^{12}$ posições!
- Com `int32` (4 bytes):

$$\text{RAM} \approx \textbf{4 TB}!$$

- **Inviável**: Estouro de memória (*Out of Memory*).

Cenário B: Lista de Adjacência

- Armazena $|V| + 2|E| = 11 \times 10^6$ elementos.
- Com ponteiros de 8 bytes:

$$\text{RAM} \approx \textbf{88 MB}$$

- **Eficiente**: Roda em qualquer notebook ou celular!

Arquitetura do TAD Grafo em Python

Adotaremos a representação em **Dicionário de Adjacência**, consolidando os conceitos de POO das Aulas 08 e 09:

```python
class Grafo:
    """
    Representacao de Grafo utilizando Dicionario de Adjacencia.
    Suporta grafos direcionados, nao-direcionados e ponderados.
    """
    def __init__(self, direcionado: bool = False):
        self._direcionado: bool = direcionado
        # Estrutura: {vertice_origem: {vertice_destino: peso}}
        self._adj: dict[str, dict[str, float]] = {}

    @property
    def eh_direcionado(self) -> bool:
        return self._direcionado

    @property
    def total_vertices(self) -> int:
        return len(self._adj)

```

Classe Grafo: Inserção de Vértices e Arestas

```
1     def adicionar_vertice(self, vertice: str) -> bool:
2         """Adiciona um novo vertice isolado caso nao exista."""
3         if vertice not in self._adj:
4             self._adj[vertice] = {}
5             return True
6         return False
7
8     def adicionar_aresta(self, u: str, v: str, peso: float = 1.0) -> None:
9         """Cria uma aresta ponderada entre u e v."""
10        # Garante a existencia dos vertices
11        self.adicionar_vertice(u)
12        self.adicionar_vertice(v)
13
14        # Insere a aresta de u para v
15        self._adj[u][v] = peso
16
17        # Se for nao-direcionado, insere a aresta simetrica de v para u
18        if not self._direcionado:
19            self._adj[v][u] = peso
20
```

Classe Grafo: Consultas e Métodos de Inspeção

```python
    def obter_vizinhos(self, vertice: str) -> list[str]:
        """Retorna a lista de vizinhos adjacentes ao vertice."""
        if vertice in self._adj:
            return list(self._adj[vertice].keys())
        return []

    def obter_peso(self, u: str, v: str) -> float | None:
        """Retorna o peso da aresta (u, v) ou None se inexistente."""
        return self._adj.get(u, {}).get(v, None)

    def grau(self, vertice: str) -> int:
        """Retorna o grau de saida ou grau total do vertice."""
        if vertice not in self._adj:
            raise KeyError(f"Vertice '{vertice}' nao encontrado no grafo.")
        return len(self._adj[vertice])

    def tem_aresta(self, u: str, v: str) -> bool:
        """Verifica a existencia de aresta direta entre u e v em tempo O(1)."""
        return (u in self._adj) and (v in self._adj[u])

```

Classe Grafo: Exibição Amigável (__str__) e Métricas

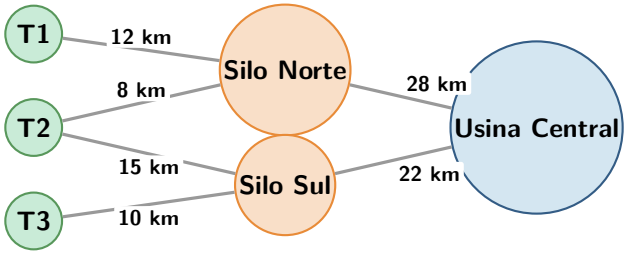
```python
    def total_arestas(self) -> int:
        """Calcula o numero total de arestas no grafo."""
        total = sum(len(vizinhos) for vizinhos in self._adj.values())
        return total if self._direcionado else total // 2

    def __str__(self) -> str:
        linhas = [f"=== Grafo ({'Direcionado' if self._direcionado else 'Nao-Direcionado'}) ==="]
        for v, vizinhos in self._adj.items():
            conexoes = ", ".join(f"{dest}(w={peso})" for dest, peso in vizinhos.items())
            linhas.append(f"  [{v}] -> {conexoes if conexoes else 'Nenhum vizinho'}")
        return "\n".join(linhas)

```

Estudo de Caso: Malha Logística de Transbordo de Safra

Cenário do Problema:
Uma cooperativa em Bebedouro gerencia o transporte diário de grãos entre três frentes de colheita (**Talhões T1, T2 e T3**), dois silos intermediários de transbordo (**Silo Norte e Silo Sul**) e a **Usina Central**.



Questão de Engenharia: Como instanciar essa malha e auditar gargalos e rotas?

Implementação do Estudo de Caso em Python

```
1  malha_agro = Grafo(direcionado=False)
2
3  # Adicionando trechos rodoviarios e distancias em km
4  malha_agro.adicionar_aresta("Talhao_1", "Silo_Norte", 12.0)
5  malha_agro.adicionar_aresta("Talhao_2", "Silo_Norte", 8.0)
6  malha_agro.adicionar_aresta("Talhao_2", "Silo_Sul", 15.0)
7  malha_agro.adicionar_aresta("Talhao_3", "Silo_Sul", 10.0)
8  malha_agro.adicionar_aresta("Silo_Norte", "Usina_Central", 28.0)
9  malha_agro.adicionar_aresta("Silo_Sul", "Usina_Central", 22.0)
10
11 print(malha_agro)
12 print(f"Total de Vertices: {malha_agro.total_vertices}")
13 print(f"Total de Arestas:  {malha_agro.total_arestas()}")
14 print(f"Vizinhos do Silo Norte: {malha_agro.obter_vizinhos('Silo_Norte')}")
15 print(f"Grau da Usina Central: {malha_agro.grau('Usina_Central')}")
16
```

Saída Gerada no Terminal

Execução do Script Python (python aula12_grafos.py)

```
=== Grafo (Nao-Direcionado) ===
  [Talhao_1] -> Silo_Norte(w=12.0)
  [Silo_Norte] -> Talhao_1(w=12.0), Talhao_2(w=8.0), Usina_Central(w=28.0)
  [Talhao_2] -> Silo_Norte(w=8.0), Silo_Sul(w=15.0)
  [Silo_Sul] -> Talhao_2(w=15.0), Talhao_3(w=10.0), Usina_Central(w=22.0)
  [Talhao_3] -> Silo_Sul(w=10.0)
  [Usina_Central] -> Silo_Norte(w=28.0), Silo_Sul(w=22.0)

Total de Vertices: 6
Total de Arestas:  6
Vizinhos do Silo Norte: ['Talhao_1', 'Talhao_2', 'Usina_Central']
Grau da Usina Central: 2
```

Exercício 1: Aplicação do Lema do Aperto de Mãos

Enunciado

É possível construir um grafo simples não-direcionado com exatamente 5 vértices cujos graus sejam respectivamente:

$$(3, 3, 3, 2, 1) \ ?$$

- Justifique matematicamente utilizando o Handshaking Lemma;
- Caso afirmativo, determine a quantidade exata de arestas $|E|$.

Tempo para reflexão: 2 minutos.

Exercício 1: Resolução Passo a Passo

Demonstração Matemática

1. **Soma dos Graus:**
$$\sum_{v \in V} \deg(v) = 3 + 3 + 3 + 2 + 1 = \mathbf{12}$$

2. **Cálculo de Arestas:**
$$\sum \deg(v) = 2|E| \implies 2|E| = 12 \implies |\mathbf{E}| = \mathbf{6}$$

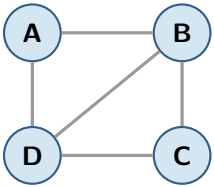
3. **Vértices Ímpares:** A lista de graus ímpares é $\{3, 3, 3, 1\}$, totalizando **4 vértices** (quantidade par, satisfazendo o corolário de Euler).

4. **Capacidade Máxima:** Para $n = 5$, o grafo completo suporta até $\frac{5 \times 4}{2} = 10$ arestas. Como $6 \leq 10$, a estrutura é viável.

5. **Conclusão: SIM**, o grafo é perfeitamente possível e possui **6 arestas**!
Exemplo de arestas: $E = \{(A, B), (A, C), (A, D), (B, C), (B, E), (C, D)\}$.

Exercício 2: Conversão entre Representações (Desafio)

Grafo Dado



Desafio para a Turma

1. Monte a **Matriz de Adjacência** 4×4.
2. Monte a **Lista de Adjacência** textual.
3. Identifique o vértice de maior grau ($\Delta(G)$).

Observe a simetria das conexões entre os nós.

Exercício 2: Resolução Passo a Passo

1. Matriz de Adjacência

$$M = \begin{pmatrix} & \mathbf{A} & \mathbf{B} & \mathbf{C} & \mathbf{D} \\ \mathbf{A} & 0 & 1 & 0 & 1 \\ \mathbf{B} & 1 & 0 & 1 & 1 \\ \mathbf{C} & 0 & 1 & 0 & 1 \\ \mathbf{D} & 1 & 1 & 1 & 0 \end{pmatrix}$$

Matriz simétrica com diagonal nula.

2. Lista de Adjacência

- $A \longrightarrow [B, D]$
- $B \longrightarrow [A, C, D]$
- $C \longrightarrow [B, D]$
- $D \longrightarrow [A, B, C]$

Vértices com Maior Grau:
$\deg(B) = 3$ e $\deg(D) = 3 \implies \Delta(G) = 3$.

Exercício 3: Métrica de Densidade de Grafo (Enunciado)

Enunciado

A **Densidade** (D) de um grafo mede a proporção entre as arestas existentes e o número máximo de arestas teoricamente possíveis.

- Para grafos não-direcionados simples:

$$D = \frac{|E|}{\frac{|V|(|V|-1)}{2}} = \frac{2|E|}{|V|(|V|-1)}$$

- Para digrafos simples: $D = \frac{|E|}{|V|(|V|-1)}$.

Tarefa: Implemente o método `densidade(self) -> float` na classe `Grafo` em Python.

Exercício 3: Implementação em Python

Solução na Classe Grafo

```python
def densidade(self) -> float:
    """
    Calcula a densidade do grafo. Retorna float no intervalo [0.0, 1.0].
    """
    n = self.total_vertices
    if n <= 1:
        return 0.0

    m = self.total_arestas()
    if self._direcionado:
        max_arestas = n * (n - 1)
    else:
        max_arestas = (n * (n - 1)) / 2

    return m / max_arestas

```

Complexidade: $O(V)$ para contar as arestas a partir do dicionário.

Exercício 4: Desafio Crítico em Big Data (Cenário)

Problema Real de Arquitetura no Campo

Uma startup do Agro monitora **500.000 sensores IoT** instalados em fazendas de citros e café.

- Cada sensor comunica-se via rádio exclusivamente com os **3 sensores mais próximos** geograficamente.
- Um estagiário sugeriu armazenar o grafo como uma Matriz Bidimensional em NumPy (`np.zeros((500000, 500000))`).

Pergunta: O que acontecerá ao executar esse código e como você, engenheiro de dados sênior, resolveria o problema?

Exercício 4: Parecer Técnico & Dimensionamento de RAM

Análise de Viabilidade Computacional

1. **Cálculo de Memória da Matriz:**

$$500.000 \times 500.000 = 2,5 \times 10^{11} \text{ células}$$

Usando `float64` (8 bytes/célula):

$$\text{RAM} = 2,5 \times 10^{11} \times 8 \text{ B} = 2 \times 10^{12} \text{ B} = \mathbf{2} \text{ Terabytes de RAM!}$$

Resultado: Falha imediata por *Out of Memory (OOM)*.

2. **Esparsidade Extrema:** Total de arestas $\approx 500.000 \times 3 = 1.500.000$. A densidade é ínfima ($D \approx 0,000006$).

3. **Solução de Engenharia:** Utilizar **Lista de Adjacência** (ou `scipy.sparse.csr_matrix`): consome menos de **50 MB de RAM**!

Resumo da Aula: O que Aprendemos Hoje?

Fundamentos Teóricos

- Origem dos Grafos com Euler em Königsberg (1736);
- Definição de $G = (V, E)$, ordem e tamanho;
- Grau de vértices e o Lema do Aperto de Mãos ($\sum \deg = 2|E|$);
- Caminhos, ciclos, DAGs e conectividade.

Engenharia & Big Data

- Matriz de Adjacência: ideal para grafos densos e consulta $O(1)$;
- Lista de Adjacência: ideal para Big Data e grafos esparsos ($O(V + E)$);
- Implementação orientada a objetos em Python com validações e tipagem.

"Grafos não são apenas estruturas de dados; são a linguagem universal para expressar relacionamentos complexos no mundo real."

Próximos Passos na Disciplina

Cronograma das Próximas Aulas

- **Aula 17:** Algoritmos de Busca em Grafos:
  - Busca em Largura (**BFS** – Breadth-First Search) e menor distância não-ponderada.
  - Busca em Profundidade (**DFS** – Depth-First Search) e detecção de ciclos.
- **Aula 18:** Ordenação Topológica e Componentes Conexos.
- **Aula 19:** Caminhos Mínimos em Grafos Ponderados (**Algoritmo de Dijkstra**).
- **Aula 20:** Árvores Geradoras Mínimas (**Prim & Kruskal**).

Atividade Prática para Casa

Baixar o arquivo `aula12_grafos.py` e adicionar à classe `Grafo` um método para remover arestas e calcular o vértice de maior grau.

Referências Bibliográficas

- WEST, Douglas B. **Introduction to Graph Theory**. 2. ed. Upper Saddle River: Prentice Hall, 2001.
- BHARGAVA, Aditya Y. **Entendendo Algoritmos: Um Guia Ilustrado para Programadores e Outros Curiosos**. São Paulo: Novatec, 2017.
- SEDGEWICK, Robert; WAYNE, Kevin. **Algorithms**. 4. ed. Addison-Wesley, 2011.
- CORMEN, Thomas H. et al. **Algoritmos: Teoria e Prática**. 3. ed. Rio de Janeiro: Elsevier, 2012.
- SZWARCFITER, Jayme Luiz. **Grafos e Algoritmos Computacionais**. Rio de Janeiro: Campus, 1984.
- DASGUPTA, Sanjoy; PAPADIMITRIOU, Christos; VAZIRANI, Umesh. **Algoritmos**. McGraw-Hill, 2008.

Bons Estudos!

Dúvidas ou Perguntas?

douglas.goncalves01@cps.sp.gov.br